

Các mô hình liên tục tuyến tính ẩn (Hidden Linear Continuous Models)

Tối ưu hoá – AI203DV01
(Optimization)
Vũ Đình Khôi

Nội dung

- Tuyến tính theo từng đoạn (Piecewise Linear)
- Khớp đường cong (Curve Fitting)
- Bài toán phân lớp mẫu nhị phân (Pattern Classification Revisited)

- Chapter 03. Serge Kruk. (2018). *Practical Python AI Projects: Mathematical Models of Optimization Problems with Google OR-Tools*. Apress

**I find that I don't understand things unless I try
to program them.**



Donald E. Knuth

Professor Emeritus at Stanford University

If you want to **master something**, teach it



-- **Richard Feynman**

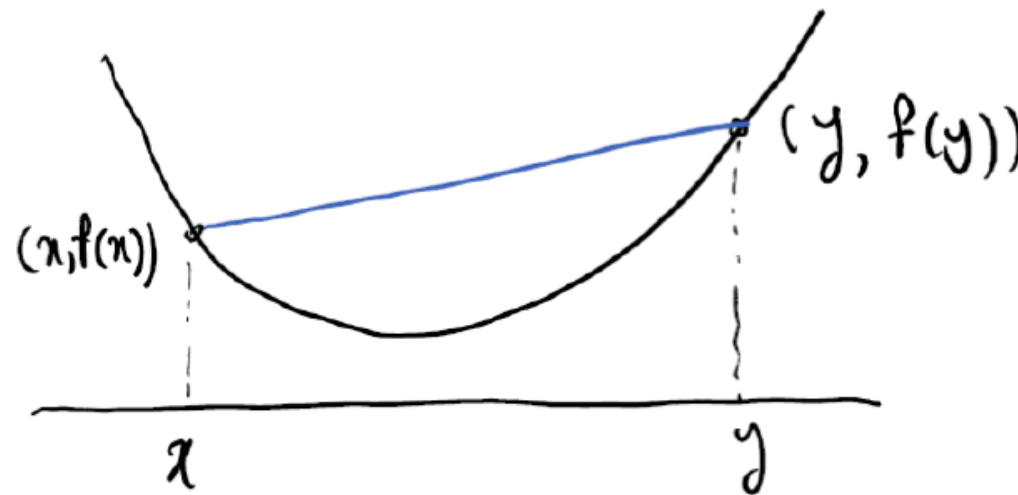
- Các công cụ giải bài toán tuyến tính liên tục (continuous linear solvers) có thể xử lý nhanh các mô hình (models) với hàng trăm ngàn biến và ràng buộc.
- Đối với các mô hình phức tạp (complex models) với chỉ vài chục biến/ràng buộc → solvers không thể tìm ra nghiệm trong thời gian thích hợp.
 - Ví dụ với các hàm piecewise linear, các solvers hiện tại (như GLPK, GLOP, CLP) không thể giải trực tiếp
- Do đó, chúng ta cố gắng biến đổi (morph) sang các dạng chuẩn tuyến tính phù hợp với solvers.

Convex function (hàm lồi)?

▪ Một số kiến thức toán học

- Đường thẳng nối 2 điểm bất kỳ trên đồ thị hàm lồi luôn nằm trên phần đồ thị.

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$



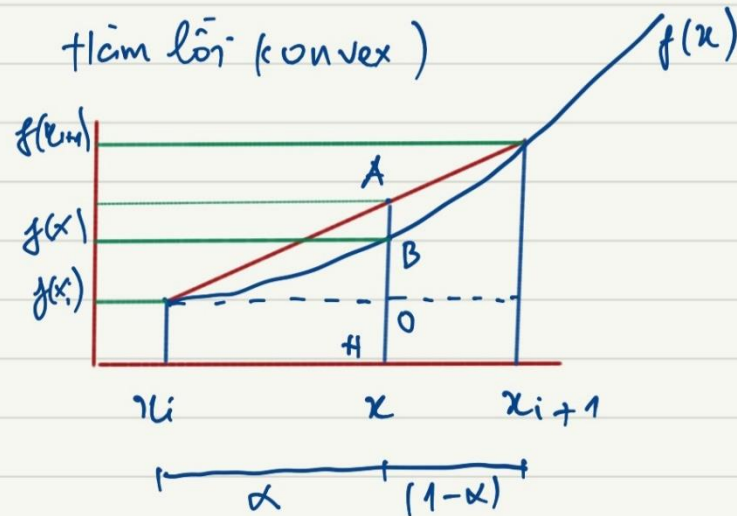
Convex function



Xem cả đoạn $[x_i, x_{i+1}]$ là 1 đơn vị (100%)

Vậy $x = ?$ Vì $\alpha = \frac{x - x_i}{x_{i+1} - x_i}$

$$\Rightarrow x = x_i + \alpha (x_{i+1} - x_i) \\ = (1 - \alpha)x_i + \alpha x_{i+1} \quad (*)$$



Đi' ý: điểm B giá trị $f(x)$ luôn nằm dưới A (do hàm lồi)
Tam giác đồng dạng:

$$\frac{OA}{f(x_{i+1}) - f(x_i)} = \frac{\alpha}{1} \quad \text{hay} \quad \frac{AH - f(x_i)}{f(x_{i+1}) - f(x_i)} = \alpha$$

$$AH = (1 - \alpha)f(x_i) + \alpha f(x_{i+1})$$

Mặt khác $BH = f(x) = f((1 - \alpha)x_i + \alpha x_{i+1})$

$$BH \leq AH : f((1 - \alpha)x_i + \alpha x_{i+1}) \leq (1 - \alpha)f(x_i) + \alpha f(x_{i+1})$$

Tổng quát: với $\alpha \in [0, 1]$ (lưu ý đặt $\beta = (1 - \alpha) \Rightarrow \alpha = (1 - \beta)$ cũng vậy)

$$f(\alpha x_i + (1 - \alpha)x_{i+1}) \leq \alpha f(x_i) + (1 - \alpha)f(x_{i+1})$$

Piecewise linear

- Xét hàm piecewise sau đây (tương tự công thức tính giá điện bậc thang, hoặc tính phí vận chuyển khi số lượng tăng → phạt nhiều tiền...)

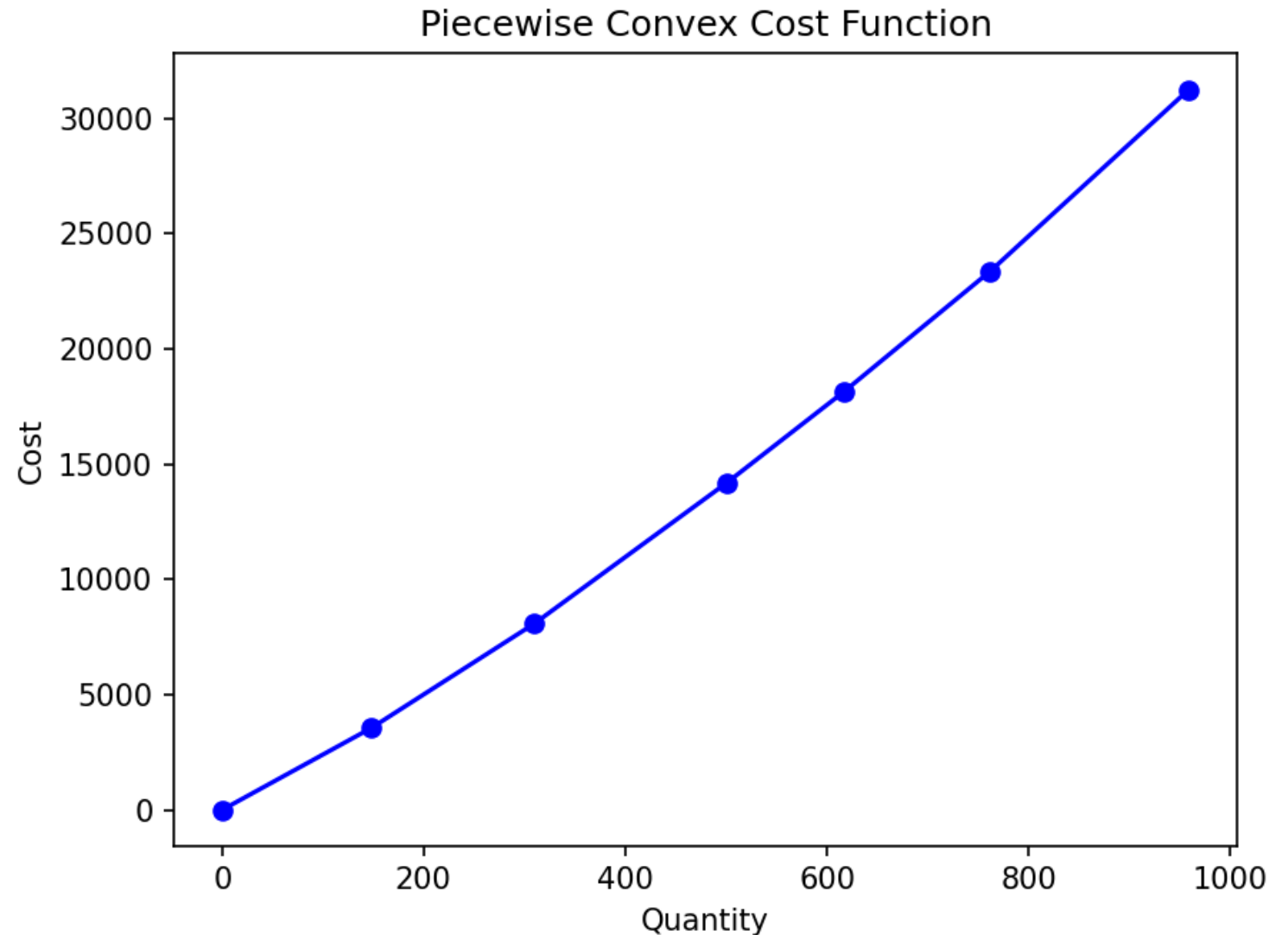
$$f(x) = \begin{cases} C_1 x & 0 \leq x \leq B_1, \\ C_1 B_1 + C_2 (x - B_1) & B_1 \leq x \leq B_2, \\ C_1 B_1 + C_2 (B_2 - B_1) + C_3 (x - B_2) & B_2 \leq x \leq B_3, \\ \dots & \end{cases}$$

Table 3-1. Example of Piecewise Function

(From	To]	Unit Cost	(Total cost	Total cost]
0	148	24	0	3552
148	310	28	3552	8088
310	501	32	8088	14200
501	617	34	14200	18144
617	762	36	18144	23364
762	959	40	23364	31244

Piecewise linear

- Hàm chi phí lồi theo từng đoạn (Piecewise convex cost function)



Piecewise linear

- Giả sử với $x = 250 \rightarrow$ dựa vào bảng công thức tính được tổng chi phí

$$C = 148 \times 24 + (250 - 148) \times 28 = 3552 + 2856 = 6408$$

- Vậy nếu biết x , có thể dùng một hàm tuyến tính để tính kết quả được không? $\rightarrow$ Tìm các hệ số của hàm tuyến tính? Ví dụ:

$$0.37 \times 3552 + 0.63 \times 8088 = 6408$$

- Với $\delta_1 = 0.37$, $\delta_2 = 0.63$, và các giá trị total cost của mỗi đoạn (bracket) là 3552, 8088

- Lưu ý công thức

$$f(x) = f(\alpha x_i + (1 - \alpha)x_{i+1}) \leq \alpha f(x_i) + (1 - \alpha)f(x_{i+1})$$

Piecewise linear: constructing a model

■ Biến quyết định

$$x \in [0, B_n]$$

- B_n là các giá trị biên (bound), tương ứng cột 1, cột 2 trong bảng.
- Giả sử có n đoạn (brackets), ta thêm các biến phụ (additional variables) δ như là các trọng số (weights) nằm giữa 2 biên (bound) của một đoạn.
 - Ví dụ, với $x = 350 \rightarrow$ cost tính toán dựa trên 3 đoạn $(0, 148], (148, 310], (310, 501]$
 $\delta_i \in [0, 1] \quad \forall i \in \{0, 1, \dots, n\}$

Piecewise linear: constructing a model

■ Các ràng buộc

$$\sum_i \delta_i = 1$$

• Suy ra

$$x = \sum_i \delta_i B_i$$

- Khi này, đối với solver, các biến δ mới thực sự là các biến quyết định (biến chúng ta cần tính để tối ưu hàm mục tiêu).

■ Hàm mục tiêu

$$\min \sum_{i=1}^n \delta_i \sum_{j=1}^i (B_j - B_{j-1}) \times C_{j-1}$$

Piecewise linear: executable model

```
1 from ortools.linear_solver import pywraplp
2
3 # Points: mảng 2D chứa gồm giá trị các Bi vào Total Cost tương ứng
4 # B: bound
5 def minimize_piecewise_linear_convex(Points, B):
6     s = pywraplp.Solver('Piecewise Linear',
7                          pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
8     n = len(Points)
9     x = s.NumVar(Points[0][0], Points[n - 1][0], 'x')
10    # d: các biến quyết định delta ( $\delta$ )
11    d = [s.NumVar(0.0, 1, 'd[%i]' % (i,)) for i in range(n)]
12    s.Add(1 == sum(d[i] for i in range(n)))
13    s.Add(x == sum(d[i] * Points[i][0] for i in range(n)))
14    s.Add(x >= B)
15    Cost = s.Sum(d[i] * Points[i][1] for i in range(n))
16    s.Minimize(Cost)
17    s.Solve()
18    # Trả về các giá trị của biến quyết định delta ( $\delta$ )
19    R = [d[i].SolutionValue() for i in range(n)]
20    return R
```

Piecewise linear: executable model

```

23 def main():
24     # [310, 8088] có nghĩa là khi Bi = 310 thì Total Cost = 8088
25     Points = [[0, 0], [148, 3552], [310, 8088], [501, 14200],
26               [617, 18144], [762, 23364], [959, 31244]]
27     B = 250
28     R = minimize_piecewise_linear_convex(Points, B)
29     print(R)

```

```

>>> %Run piecewise_linear.py
[0.0, 0.37037037037037046, 0.6296296296296295, 0.0, 0.0, 0.0, 0.0]

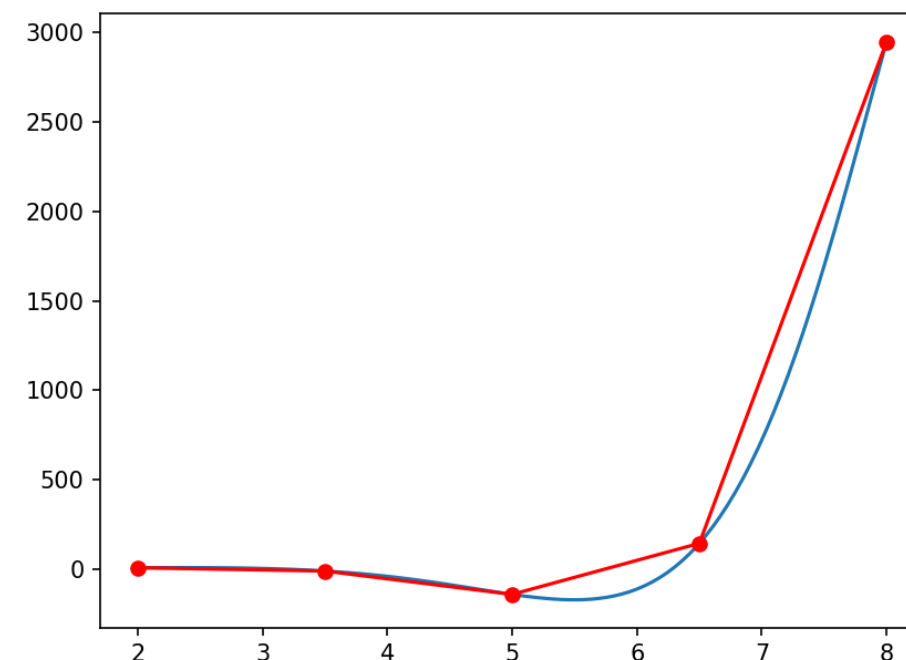
```

Table 3-2. Optimal Solution to Convex Piecewise Objective with $x \geq 250$

Interval	0	1	2	3	4	5	6	Solution
δ_i	0.0	0.3704	0.6296	0.0	0.0	0.0	0.0	$\sum \delta = 1.0$
x_i	0	148	310	501	617	762	959	$x = 250.0$
$f(x_i)$	0	3552	8088	14200	18144	23364	31244	Cost=6408

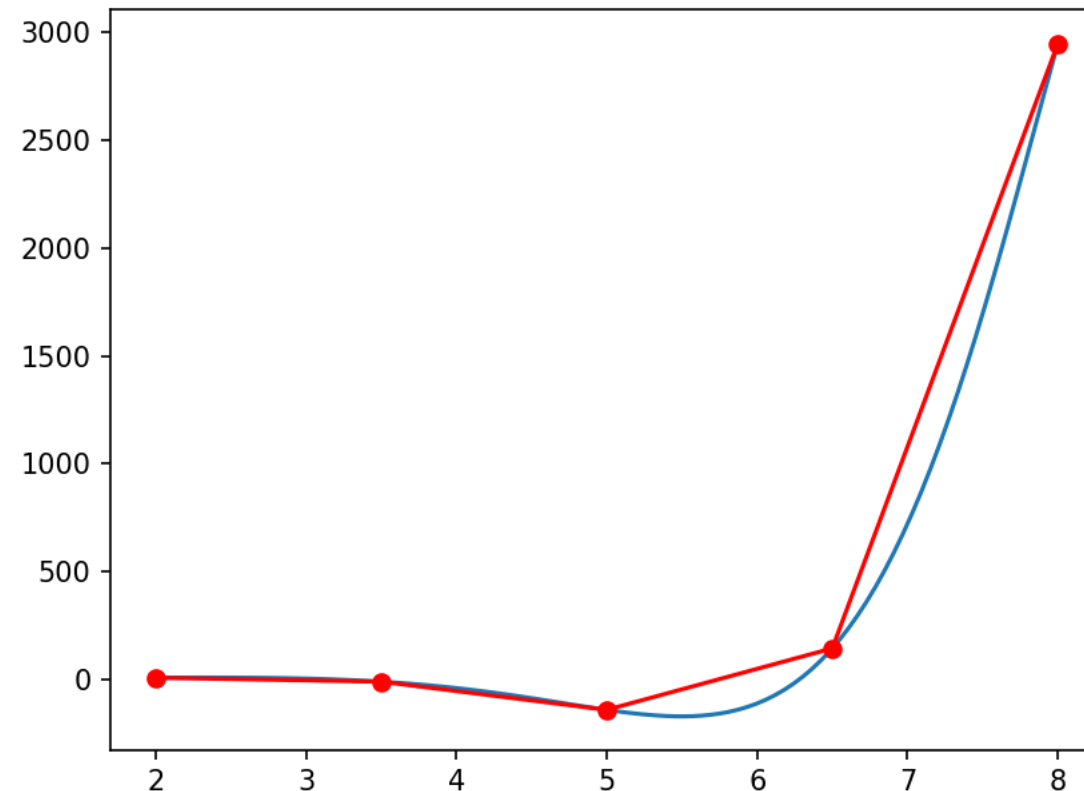
Non-Linear Function Minimization via Linear Approximations

- Tối thiểu hàm phi tuyến bằng xấp xỉ tuyến tính
 - Vì có thể giải các bài toán tối ưu với các hàm tuyến tính từng đoạn (piecewise linear functions) → có thể dùng cách tiếp cận này để xấp xỉ các hàm phi tuyến lồi (convex non-linear functions).
- Ví dụ, tìm cực tiểu hàm phi tuyến $f(x) = \sin(x)e^x$ trên đoạn $[2, 8]$
- Quan sát
 - Giá trị đường $f(x)$ màu xanh nhỏ hơn bằng (nằm dưới) các đoạn tuyến tính đỏ
→ tìm cực tiểu các xấp xỉ tuyến tính theo đoạn



Non-Linear Function Minimization via Linear Approximations

```
1 # Non-Linear Function Minimization via Linear Approximations
2 # Tìm cực tiểu hàm phi tuyến bằng phương pháp xấp xỉ tuyến tính theo đoạn
3 # Các hàm số phi tuyến được xấp xỉ bằng các đoạn tuyến tính
4
5 from piecewise_linear import minimize_pieewise_linear_convex
6
```



Non-Linear Function Minimization via Linear Approximations

```
8 # func: hàm phi tuyến cần tìm cực tiểu
9 def minimize_non_linear(func, left, right, precision):
10     n = 5 # 5 điểm trên đồ thị hàm số --> 4 đoạn tuyến tính
11     x = 0.0
12     while right - left > precision:
13         dta = (right - left) / (n - 1.0) # delta: khoảng cách giữa các điểm
14         # points: danh sách các điểm/đoạn trên đồ thị hàm số
15         points = [(left + dta * i, func(left + dta * i)) for i in range(n)]
16         # G: giá trị của các biến quyết định delta ( $\delta$ )
17         G = minimize_piecewise_linear_convex(points, left)
18         # x: giá trị x tại điểm cực tiểu
19         x = sum([G[i] * points[i][0] for i in range(n)])
20         # left, right: giới hạn mới của đoạn cần tìm cực tiểu
21         # G[i] > 0: điểm i là điểm cuối cùng của đoạn tuyến tính
22         # index của các đoạn có G[i] > 0
23         indices = [i - 1 for i in range(n) if G[i] > 0]
24         left = points[max(0, indices[0])][0]
25         # vết gọn: left = points[max(0, [i - 1 for i in range(n) if G[i] > 0][0])][0]
26         right = points[min(n - 1, [i + 1 for i in range(n - 1, 0, -1) if G[i] > 0][0])][0]
27     return x
```

Non-Linear Function Minimization via Linear Approximations

```
30 def main():
31     # Tìm cực tiểu hàm  $f(x)=\sin(x)*e^x$  trong khoảng [2, 8]
32     # với độ chính xác 0.05
33     def func(x):
34         from math import sin, exp
35         return sin(x) * exp(x)
36
37     left = 2
38     right = 8
39     precision = 0.05
40     x = minimize_non_linear(func, left, right, precision)
41     print('Min at x: {:.1f}'.format(x))
42     print('Min value: {:.1f}'.format(func(x)))
43
```

```
>>> %Run min_non_linear.py
Min at x: 5.5
Min value: -172.6
```

Non-Convex Piecewise Linear

- Đối với hàm tuyến tính từng đoạn phi lồi thì sao?
 - Bài toán mua với số lượng càng nhiều → càng giảm giá
- Không thể giải với hàm `minimize_piecewise_linear_convex()`

Table 3-6. *Example of Non-Convex Piecewise Function*

(From	To]	Unit cost	(Total cost	Total cost]
0	194	18	0	3492
194	376	16	3492	6404
376	524	14	6404	8476
524	678	13	8476	10478
678	820	11	10478	12040
820	924	6	12040	12664

Non-Convex Piecewise Linear

- Sẽ trở lại
 - trong section 7.2 chapter 7

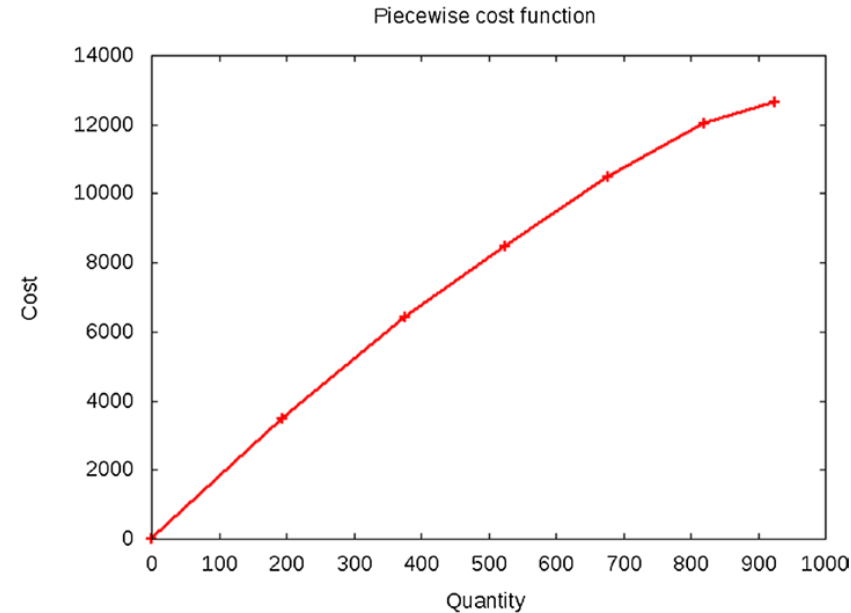
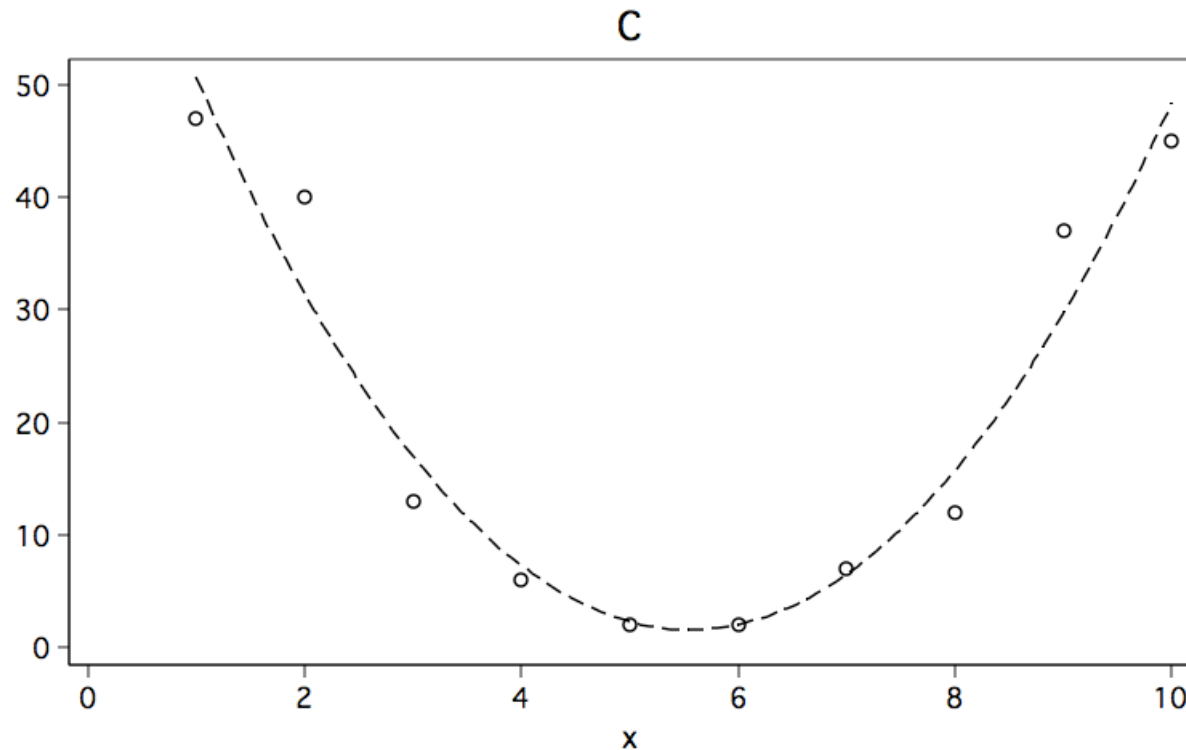


Table 3-7. *Incorrect Solution to Non-Convex Objective with $x \geq 250$*

Interval	0	1	2	3	4	5	6	Solution
δ_i	0.7294	0.0	0.0	0.0	0.0	0.0	0.2706	$\sum \delta = 1.0$
x_i	0	194	376	524	678	820	924	$x = 250.0$
$f(x_i)$	0	3492	6404	8476	10478	12040	12664	Cost=3426

Curve fitting

- Bài toán rất thông dụng, từ tập hợp các điểm dữ liệu → tìm đường đa thức phù hợp (gần khớp nhất) với tập điểm dữ liệu (Polynomial curves fitting).
 - Đơn giản: hồi quy tuyến tính, bậc hai, bậc ba, loga (linear/quadratic/cubic/logarithmic regression)...



Curve fitting

- Tìm đường cong bậc hai khớp/phù hợp với tập điểm dữ liệu
 - $f(t) = a_2t^2 + a_1t + a_0$
 - → tìm các hệ số (coefficients) a_i
- Khoảng cách Euclid
 - $(f_i - f(t_i))^2$
- Hàm mục tiêu/mất mát/chi phí (objective/lost/cost function) theo tiếp cận Least-squares

$$\min \sum_n (f_i - f(t_i))^2$$

t_i	f_i
0.1584	0.0946
0.8454	0.2689
2.1017	5.8285
3.1966	14.8898
4.056	25.6134
4.9931	38.3952
5.8574	43.5065
7.1474	91.3715
8.1859	119.075
9.0349	115.7737

Curve fitting: objective function

- Hàm mục tiêu/mất mát/chi phí (objective/lost/cost function) theo tiếp cận Least-squares

$$\min \sum_n (f_i - f(t_i))^2$$

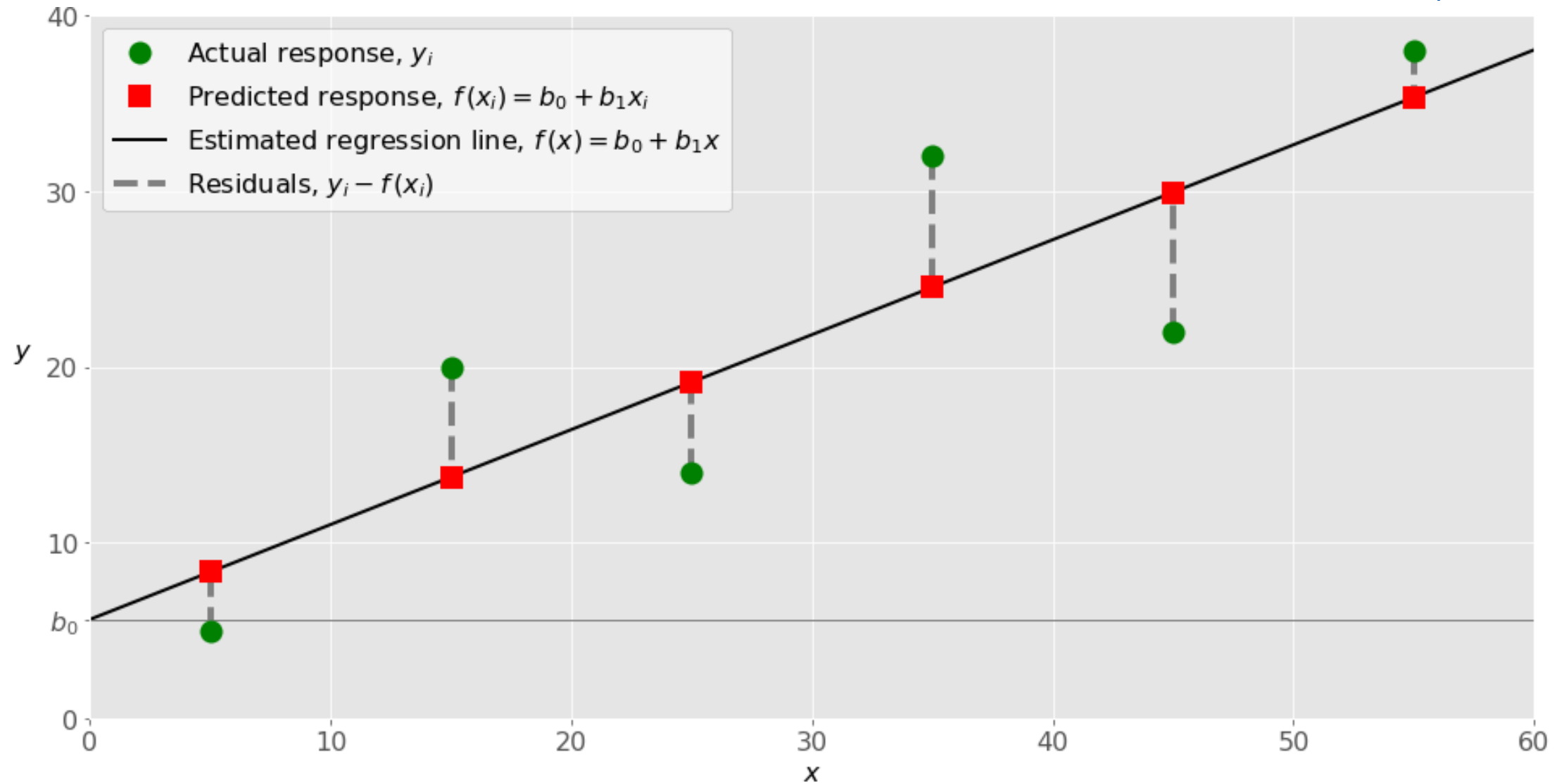
- Theo giá trị tuyệt đối (absolute values)

$$\min \sum_n |f_i - f(t_i)|$$

- Theo phương pháp min-max (nghĩa là tìm cách cực tiểu độ lệch (deviation) có giá trị lớn nhất) hay Chebyshev's criterion
 - Tất cả các độ lệch/sai số (deviation/error/residual) phải nằm trong giá trị cực đại nào đó

$$\min \max_n |f_i - f(t_i)|$$

Curve fitting



Curve fitting: constructing a model

- Hàm mục tiêu (objective function)
- Tổng quát, xác định đa thức bậc k theo biến t. Cần xác định các hệ số $a_0, a_1, \dots, a_k$ để cực tiểu tổng các độ lệch (sum of deviation) hoặc độ lệch lớn nhất (largest deviation) giữa các điểm dữ liệu và đường đa thức.
- Gọi e_i là độ lệch hay sai số (error) của từng điểm dữ liệu (t_i, f_i)
- Hàm mục tiêu theo tổng các độ lệch

$$\min \sum_i e_i$$

- Hàm mục tiêu theo độ lệch lớn nhất

$$\min \max_n e_i$$

Curve fitting: constructing a model

■ Lưu ý:

- Biểu thức min-max không phù hợp với linear programming, vì chỉ có thể tối ưu hóa theo min hoặc max nhưng không thể cùng lúc cả hai.
- Chỉ có thể giải một hàm mục tiêu, không thể là tập hợp các hàm mục tiêu.

- Do đó, phải tìm cách chuyển cách mục tiêu (cả min lẫn max) thành các ràng buộc (constraints)
- Cho biến mới e và tập các bất đẳng thức (inequalities) để biểu diễn độ lệch lớn nhất như sau

$$e_i \leq e \quad \forall i \in [1, n]$$

- Bởi vì e là cận trên (upper bound) của tất cả các độ lệch, do đó mục tiêu
 $\min e$

Curve fitting: constructing a model

- Các ràng buộc

$$|f_i - f(t_i)| \leq e$$

- Bất đẳng thức (có trị tuyệt đối) trên không phải tuyến tính, do đó tìm cách chuyển sang dạng tuyến tính. Có ít nhất 2 cách khác nhau để xử lý
- Cách 1: Chuyển thành 2 bất đẳng thức và 2 cận trên (Double the inequalities and bound the deviations)

$$f_i - f(t_i) \leq e$$

$$-f_i + f(t_i) \leq e$$

- Cách này không tính được độ lệch của mỗi điểm
- *Giải thích trị tuyệt đối (xem slide kế tiếp)*

Curve fitting: constructing a model

7.2

Linear Programming 1: Geometric Solutions

Consider using the Chebyshev criterion to fit the model $y = cx$ to the following data set:

x	1	2	3
y	2	5	8

- The optimization problem that determines the parameter c to minimize the largest absolute deviation $r_i = |y_i - y(x_i)|$ (residual or error) is the linear program

Minimize r

subject to

$$\left. \begin{aligned} r - (2 - c) &\geq 0 && \text{(constraint 1)} \\ r + (2 - c) &\geq 0 && \text{(constraint 2)} \\ r - (5 - 2c) &\geq 0 && \text{(constraint 3)} \\ r + (5 - 2c) &\geq 0 && \text{(constraint 4)} \\ r - (8 - 3c) &\geq 0 && \text{(constraint 5)} \\ r + (8 - 3c) &\geq 0 && \text{(constraint 6)} \end{aligned} \right\}$$

(7.2)

Vũ Đình Khôi 5:33 PM

Xem như chuyển từ bài toán Hồi quy tuyến tính/Khớp đường cong (Linear Regression/Curve Fitting) sang bài toán Quy hoạch tuyến tính (Linear Programming)

Add a reply

Vũ Đình Khôi 5:40 PM

Do:

$|a| = a$ nếu $a \geq 0$

$|a| = -a$ nếu $a < 0$

Add a reply

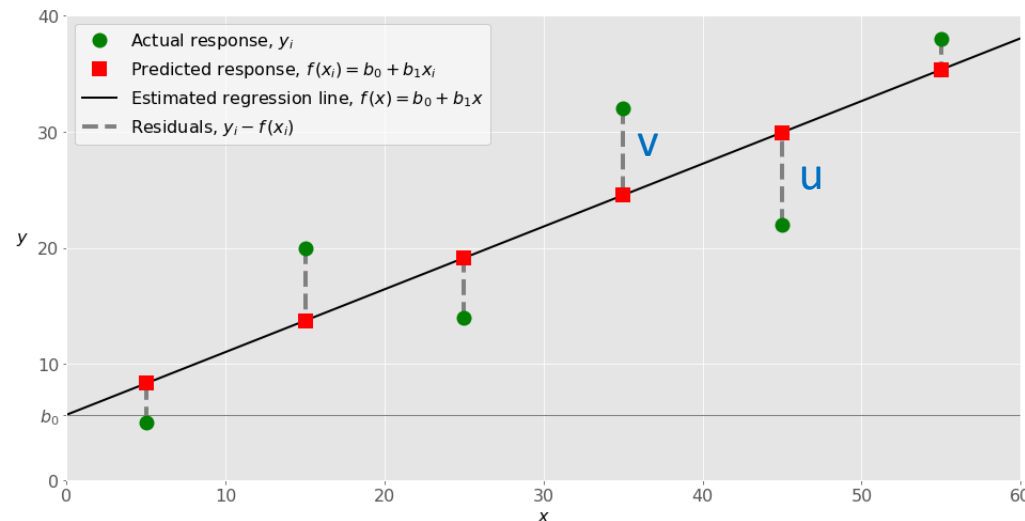
Curve fitting: constructing a model

■ Cách 2: Sử dụng 2 biến phụ và tính độ lệch từng điểm dữ liệu (Double the variables and find each deviation)

- Gọi u_i là độ lệch nằm dưới hàm $f(t_i)$
- Gọi v_i là độ lệch nằm trên hàm $f(t_i)$
- u_i, v_i là các giá trị không âm (non-negative)
- Đối với mỗi điểm dữ liệu, bất đẳng thức sẽ thành đẳng thức (equality)

$$f(t_i) - u_i + v_i = f_i$$

- Nhận xét đẳng thức trên luôn chỉ có một (u hoặc v) khác không (non-zero) và biến còn lại bằng không (zero).



Curve fitting: constructing a model

■ Cách 2: chia thành 2 trường hợp

- Muốn tối thiểu tổng độ lệch (minimize the sum of deviations)

$$\min \sum_n (u_i + v_i)$$

- Tối thiểu độ lệch tối đa (minimize the maximum deviation), cần thêm 2 bất đẳng thức

$$u_i \leq e$$

$$v_i \leq e$$

$$\min e$$

- e là cận trên (upper bound) của 2 biến u, v

Curve fitting: executable model

```
1 # Polynomial Curve Fitting Model
2 # Sử dụng Linear Programming để giải bài toán Curve Fitting
3 # Tìm đường đa thức khớp bậc n khớp với tập điểm dữ liệu
4 from ortools.linear_solver import pywraplp
5 from my_or_tools import ObjVal, SolVal
6
7
8 def solve_curve_fitting(D, degree=1, objective=0):
9     s = pywraplp.Solver('Polynomial Curve Fitting',
10                        pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
11
12     n = len(D)
13     b = s.infinity() # bound (giới hạn/giá trị biên)
14     # Các biến quyết định hệ số a của đa thức
15     # Số hệ số a = bậc đa thức + 1 (vì có a0)
16     a = [s.NumVar(-b, b, 'a[%i]' % i) for i in range(degree + 1)]
17     # Các biến phụ u, v
18     u = [s.NumVar(0.0, b, 'u[%i]' % i) for i in range(n)]
19     v = [s.NumVar(0.0, b, 'v[%i]' % i) for i in range(n)]
20     # Biến giá trị cực đại của độ lệch (deviation)
21     e = s.NumVar(0.0, b, 'e')
```

Curve fitting: executable model

```
21 # Các ràng buộc
22 for i in range(n):
23     s.Add(u[i] - v[i] ==
24           D[i][1] - sum(a[j] * D[i][0] ** j for j in range(degree + 1)))
25 for i in range(n):
26     s.Add(u[i] <= e)
27     s.Add(v[i] <= e)
28 # Hàm mục tiêu (xét cho 2 cách tiếp cận sử dụng trị tuyệt đối hay min-max)
29 if objective == 0: # sum fit
30     Cost = sum(u[i] + v[i] for i in range(n))
31 else: # max fit
32     Cost = e
33 s.Minimize(Cost)
34 status = s.Solve()
35 return status, ObjVal(s), SolVal(a)
36
```


Curve fitting: executable model

```
38 def main():
39     D = [[0.1584, 0.0946],
40          [0.8454, 0.2689],
41          [2.1017, 5.8285],
42          [3.1966, 14.8898],
43          [4.056, 25.6134],
44          [4.9931, 38.3952],
45          [5.8574, 43.5065],
46          [7.1474, 91.3715],
47          [8.1859, 119.075],
48          [9.0349, 115.7737]]
49
50     status, obj_val, sol_val = solve_curve_fitting(D, degree=2, objective=0)
51     print('Status =', status)
52     print('Objective value (min of sum (u + v)) =', obj_val)
53     print('Solution a[] =', sol_val)
54
```

Curve fitting: executable model

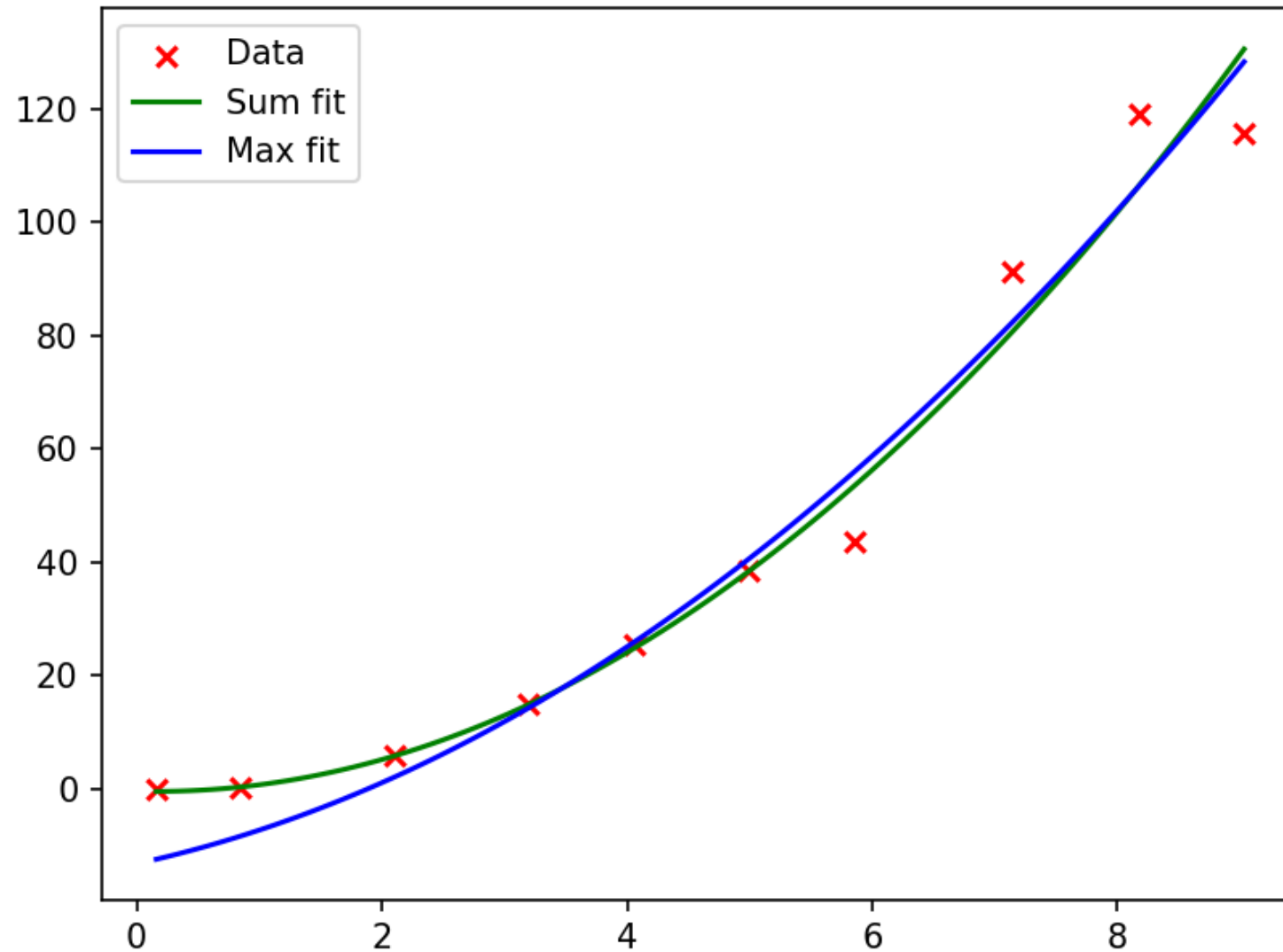
```
50 status, obj_val, sol_val = solve_curve_fitting(D, degree=2, objective=0)
51 print('Status =', status)
52 print('Objective value (min of sum (u + v)) =', obj_val)
53 print('Solution a[] =', sol_val)
54
55 # Vẽ đồ thị các điểm dữ liệu và đường cong đa thức
56 import matplotlib.pyplot as plt
57 import numpy as np
58 X = np.array(D)[: , 0]
59 Y = np.array(D)[: , 1]
60 plt.scatter(X, Y, marker='x', color='red')
61
62 # Phương pháp sum fit
63 a = sol_val
64 X = np.linspace(min(X), max(X), 100)
65 Y = sum([a[i] * X ** i for i in range(len(a))])
66 plt.plot(X, Y, color='green')
67
```

Curve fitting: executable model



```
>>> %Run curve_fitting.py  
Status = 0  
Objective value (min of sum (u + v)) = 49.24090544016789  
Solution a[] = [-0.39320545554284503, -0.5345761374298995, 1.662890313149744]  
Status = 0  
Objective value (min of max deviation) = 12.50106076954847  
Solution a[] = [-13.185510708337027, 4.726611191089643, 1.2098066412210806]
```

Curve fitting: executable model



Pattern Classification Revisited

- Trở lại bài toán phân lớp [nhị phân] tập mẫu
 - Tìm một siêu phẳng (hyperplane) nhằm tách biệt hai tập điểm dữ liệu một cách tối đa, nghĩa là hyperplane cách đều 2 tập điểm dữ liệu.
 - Siêu phẳng có thể là đường thẳng (không gian 2-D), mặt phẳng (trong không gian 3-D), từ 4-D trở lên có thể tưởng tượng một siêu phẳng $Ax = b$
- Một trong các cách là tối đa khoảng cách tối thiểu từ tập huấn luyện đến siêu phẳng (maximize the minimum distance from the training set to the separating hyperplane)
 - Cách tiếp cận này được gọi là Tối đa các khoảng cách lề (maximizing the margins)
 - Ý tưởng các điểm dữ liệu của 2 lớp A, B càng cách xa đường phân tuyến càng tốt → kết quả phân lớp chính xác cao.

Pattern Classification Revisited

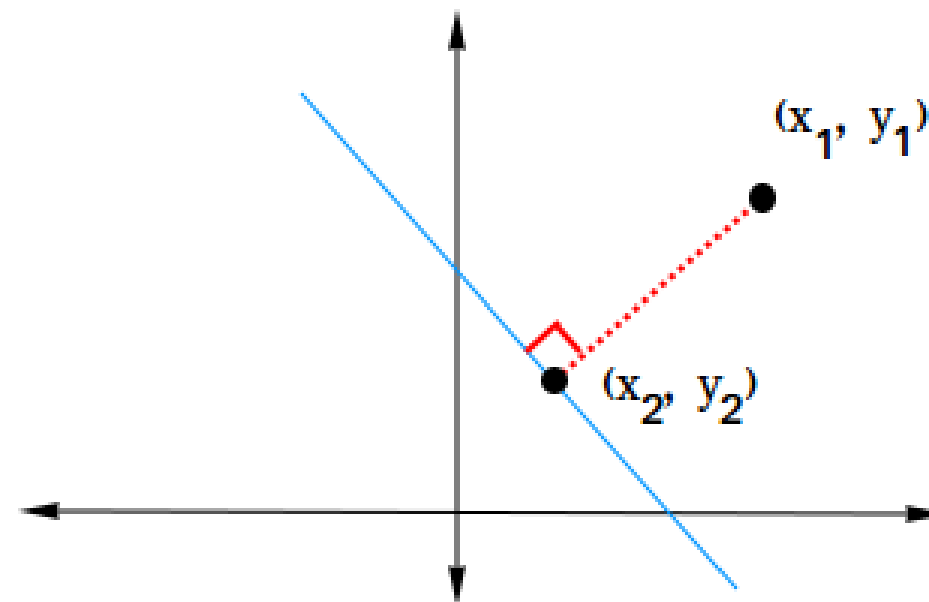
- Siêu phẳng (hyperplane) có dạng

$$\sum_{i=1}^n a_i x_i = a_0$$

- Khoảng cách (distance) từ điểm dữ liệu x' đến siêu phẳng?

$$d = \frac{|\sum a_i x'_i - a_0|}{\sqrt{\sum a_i^2}}$$

- Đây là khoảng cách vuông góc/ngắn nhất (đừng nhầm với độ lệch deviation)
- Công thức trên tính dựa trên vector pháp tuyến (kiến thức phổ thông)



Pattern Classification Revisited

- Vì ta muốn tối đa giá trị tối thiểu d trên toàn bộ điểm dữ liệu

- Chỉ xét phần tử số (numerator), phần mẫu số (denominator) $\sqrt{\sum a_i^2}$ không ảnh hưởng.
- Sử dụng kỹ thuật thêm 2 biến mới u, l để chuyển phần trị tuyệt đối $|\sum a_i x_i' - a_0|$ thành 3 ràng buộc (a triplet of constraints) sau:

$$\sum a_i x_i' - u + l = a_0$$

$$u \geq e$$

$$l \geq e$$

- e là cận dưới (lower bound) của 2 biến u, l . Tại sao?
- Vì ta muốn 2 tập dữ liệu càng cách xa hyperplane càng tốt? Vì khi này việc phân lớp sẽ chính xác hơn.

Pattern Classification Revisited: executable model

```

10 def solve_margins_classification(A, B):
11     n = len(A[0])
12     ma, mb = len(A), len(B)
13     s = pywraplp.Solver('Classification with maximizing the margins',
14                         pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
15     # Các biến quyết định
16     ua = [s.NumVar(0, 99, '') for _ in range(ma)]
17     la = [s.NumVar(0, 99, '') for _ in range(ma)]
18     ub = [s.NumVar(0, 99, '') for _ in range(mb)]
19     lb = [s.NumVar(0, 99, '') for _ in range(mb)]
20     a = [s.NumVar(-99, 99, '') for _ in range(n + 1)]
21     e = s.NumVar(0, 99, 'e')
22     # Các ràng buộc, lưu ý: a[n] chính là a0 trong slide/textbook
23     #  $a_0 * x_0 + a_1 * x_1 + \dots = a_n$ 
24     for i in range(ma):
25         # Điều kiện/ràng buộc để điểm dữ liệu x thuộc lớp A
26         s.Add(0 >= a[n] + 1 - s.Sum(a[j] * A[i][j] for j in range(n)))
27         # Điều kiện/ràng buộc khoảng cách từ điểm x đến hyperplane
28         s.Add(a[n] == s.Sum(a[j] * A[i][j] - ua[i] + la[i] for j in range(n)))

```


Pattern Classification Revisited: executable model

```
29         # e là cận dưới của u, l
30         s.Add(ua[i] >= e)
31         s.Add(la[i] >= e)
32     for i in range(mb):
33         s.Add(0 >= s.Sum(a[j] * B[i][j] for j in range(n)) - a[n] + 1)
34         s.Add(a[n] == s.Sum(a[j] * B[i][j] - ub[i] + lb[i] for j in range(n)))
35         s.Add(ub[i] >= e)
36         s.Add(lb[i] >= e)
37     s.Minimize(e)
38     status = s.Solve()
39     return status, SolVal(a)
40
```

Pattern Classification Revisited: executable model



```
>>> %Run pattern_classification_2.py
```

```
Status = 0
```

```
Solution = [-34.19527178658282, 55.57575407882255, 66.52346465574786]
```

